



APIs e integrações - conceitos, tipos, ferramentas, exemplos

Prof.º Denilce Veloso

Disciplina: Programação Web

Mateus Marcondes Tognoli 0030482511007

Sorocaba

Março/2026

1. Introdução

Ao longo dos anos, a evolução dos sistemas digitais aumentou a necessidade de comunicação entre aplicações, serviços e plataformas diferentes. Um aplicativo de comércio eletrônico, por exemplo, pode precisar consultar um serviço de pagamento, calcular um frete, enviar uma mensagem ao cliente e atualizar o estoque. Para que esses componentes consigam trocar informações de maneira organizada, são utilizadas interfaces de programação de aplicações, conhecidas pela sigla API (Application Programming Interface).

Uma API pode ser entendida como um conjunto de regras e recursos que define como um software pode interagir com outro software. Na Web, as APIs normalmente utilizam protocolos como HTTP e formatos de dados como JSON, permitindo que aplicações desenvolvidas em linguagens e ambientes diferentes troquem informações sem que uma precise conhecer toda a estrutura interna da outra.

As integrações, por sua vez, correspondem ao processo de conectar sistemas para que dados, funções ou eventos possam circular entre eles. Nesse contexto, APIs são uma das principais formas de integração, mas não são a única. Também podem ser utilizados webhooks, filas de mensagens, gateways de API, bibliotecas de integração e outros mecanismos. Com isso, o objetivo desta pesquisa é apresentar os principais conceitos relacionados a APIs e integrações, seus tipos, ferramentas utilizadas no desenvolvimento e testes, exemplos de uso e aspectos de segurança e boas práticas.

2. Conceitos Fundamentais

2.1. O que é uma API?

API é uma interface que estabelece um contrato de comunicação entre componentes de software. Em vez de um programa acessar diretamente a implementação interna de outro, ele utiliza as operações disponibilizadas pela interface. Isso reduz o acoplamento entre sistemas e permite que cada componente seja desenvolvido, mantido ou substituído de forma mais independente.

No desenvolvimento Web, uma API pode disponibilizar endpoints, que são endereços associados a determinados recursos ou operações. Um cliente envia uma requisição e o servidor processa essa solicitação, retornando uma resposta. Em APIs HTTP, os métodos GET, POST, PUT, PATCH e DELETE são frequentemente utilizados para representar operações de leitura, criação e alteração de recursos.

2.2. API, endpoint, requisição e resposta

Alguns termos são fundamentais para compreender o funcionamento de uma API. Endpoint pode ser definido como o ponto de acesso disponibilizado pelo serviço, enquanto a resposta é descrita como a mensagem retornada pelo servidor. Já a requisição é a mensagem enviada pelo cliente, podendo conter método HTTP, URL, cabeçalhos, parâmetros e corpo. A resposta normalmente apresenta um código de status, cabeçalhos e dados no corpo da mensagem.

Em uma API REST, por exemplo, uma aplicação poderia disponibilizar o endpoint `"/usuarios/15"`. Uma requisição GET para esse endereço poderia solicitar os dados do usuário 15, enquanto uma requisição PUT poderia atualizar informações desse recurso, desde que a API tenha definido essa operação. A ideia é que o consumidor da API não precise conhecer a lógica interna do servidor para utilizá-la.

2.3. Formatos de dados

O JSON (JavaScript Object Notation) é um dos formatos mais utilizados em APIs Web por ser simples, legível e facilmente processado por diferentes linguagens. Outro formato bastante utilizado é o XML, especialmente em integrações mais antigas ou em serviços baseados em SOAP. A escolha do formato acontece baseada nos requisitos da aplicação, da tecnologia empregada e da compatibilidade necessária entre os sistemas.

3. Tipos de APIs

3.1. APIs públicas, privadas e de parceiros

As APIs podem ser classificadas de diferentes formas, sendo uma das principais classificações baseada no público que possui autorização para utilizá-las, permitindo compreender melhor a finalidade, o nível de acesso e as regras de utilização de cada tipo de API. APIs públicas, também chamadas de abertas, são disponibilizadas para desenvolvedores externos, geralmente seguindo regras de autenticação, limites de uso e políticas específicas, enquanto as APIs privadas são utilizadas internamente por uma organização para integrar sistemas, aplicações e serviços próprios. Já as APIs de parceiros são disponibilizadas para empresas ou organizações autorizadas, geralmente mediante contratos, credenciais e regras de acesso.

3.2. APIs REST

O REST (Representational State Transfer) é um estilo arquitetural para sistemas distribuídos, descrito por Roy Fielding no ano de 2000. O REST utiliza restrições arquiteturais que favorecem escalabilidade, interfaces uniformes, independência entre componentes e uso de intermediários. É importante destacar que o REST é um estilo arquitetural, e não um protocolo específico.

Na prática, APIs REST costumam utilizar HTTP e representar recursos por meio de URLs. Um sistema pode, por exemplo, disponibilizar `/produtos` para representar uma coleção de produtos e `/produtos/25` para representar um produto específico. A combinação entre recursos, métodos HTTP e códigos de status permite construir interfaces relativamente simples e interoperáveis.

Por outro lado, o REST pode levar a situações de over-fetching ou under-fetching, problemas que ocorrem quando o cliente recebe mais ou menos dados do que realmente precisa, exigindo múltiplas requisições para montar uma tela completa.

3.3. APIs SOAP

O SOAP (Simple Object Access Protocol) é um framework de mensagens estruturadas utilizado em ambientes distribuídos. O W3C publicou a especificação SOAP 1.2 como recomendação, definindo um modelo de mensagens baseado em XML e mecanismos de extensão e processamento. O SOAP é comum em ambientes corporativos nos quais contratos formais, padrões legados e interoperabilidade entre plataformas são requisitos importantes.

Em contrapartida, o SOAP tende a ser mais verboso que alternativas como REST, e sua curva de aprendizado é considerada mais alta, o que reduziu sua popularidade em novos projetos Web.

3.4. APIs GraphQL

O GraphQL é uma abordagem de consulta a APIs na qual o cliente especifica os dados de que precisa, em vez de depender exclusivamente de respostas fixas definidas pelo servidor. Essa característica pode ser útil quando diferentes clientes precisam de conjuntos diferentes de informações. Em comparação com uma API REST tradicional, GraphQL pode reduzir situações em que o cliente recebe dados desnecessários ou precisa realizar várias requisições para montar uma tela.

Apesar dessa flexibilidade, o GraphQL exige mais esforço de configuração no servidor e pode dificultar estratégias simples de cache, algo que o REST, por exemplo, resolve de forma mais direta.

3.5. APIs WebSocket

WebSocket é apropriado para cenários que exigem comunicação bidirecional e contínua entre cliente e servidor, como chats, painéis em tempo real e aplicações colaborativas. Diferentemente do modelo tradicional de requisição e resposta usado por muitas APIs HTTP, WebSocket permite que mensagens sejam trocadas nos dois sentidos por uma conexão persistente. Serviços como o Amazon API Gateway oferecem suporte tanto a APIs HTTP/REST quanto a APIs WebSocket. Essa comunicação contínua, no entanto, exige mais recursos do servidor para manter

conexões abertas, tornando o WebSocket menos indicado para cenários onde não há necessidade real de tempo real.

4. Integrações entre Sistemas

4.1. O que significa integrar sistemas?

A integração de sistemas pode ser definida como o processo de fazer aplicações diferentes trabalharem juntas. Em uma empresa, por exemplo, um sistema de vendas pode precisar enviar dados para o sistema financeiro, consultar o estoque e acionar uma transportadora. A integração pode ser síncrona, quando um sistema solicita uma informação e espera uma resposta, ou assíncrona, quando uma mensagem ou evento é enviado para ser processado posteriormente.

4.2. Webhooks

Webhook é um mecanismo no qual um serviço envia uma notificação para uma URL previamente configurada quando determinado evento acontece. Ele é especialmente útil para integrações orientadas a eventos. Por exemplo, um sistema de pagamentos pode avisar automaticamente uma aplicação quando um pagamento foi confirmado, evitando que a aplicação precise consultar o serviço repetidamente.

A documentação da Stripe apresenta esse modelo: quando eventos ocorrem, a plataforma pode enviar dados em tempo real para um endpoint HTTPS de webhook, normalmente em formato JSON. Esse tipo de integração é adequado para eventos assíncronos, como confirmação de pagamentos e outras mudanças de estado.

4.3. API Gateway

Um API Gateway funciona como uma porta de entrada para APIs e serviços de backend. Ele pode centralizar funções como roteamento, autenticação, controle de acesso, monitoramento, limitação de tráfego e gerenciamento de versões. A AWS descreve seu API Gateway como um serviço para criar, publicar, manter, monitorar e proteger APIs REST, HTTP e WebSocket.

4.4. Integração por polling

Polling é uma estratégia na qual uma aplicação consulta periodicamente outra aplicação para verificar se existem novos dados. Embora seja simples de implementar, pode gerar requisições desnecessárias quando não há novidades. Por esse motivo, quando o sistema oferece suporte a eventos e webhooks, o modelo orientado a eventos pode ser mais eficiente para determinados casos.

5. Ferramentas para desenvolvimento e integração

5.1. Postman

Postman é uma ferramenta amplamente utilizada para criar, enviar e testar requisições de APIs. Ela permite organizar coleções de requisições, configurar parâmetros e cabeçalhos, testar respostas e automatizar verificações. Ela é muito útil tanto para desenvolvimento quanto para validação de integrações antes que elas sejam incorporadas a uma aplicação.

5.2. Insomnia

Insomnia é outra ferramenta utilizada para trabalhar com APIs, permitindo criar e executar requisições HTTP e organizar ambientes de desenvolvimento. Ferramentas desse tipo ajudam a identificar problemas de autenticação, parâmetros, formatos de resposta e códigos de erro sem que seja necessário testar tudo diretamente pela interface final da aplicação.

5.3. OpenAPI

OpenAPI é uma especificação independente de linguagem para descrever APIs HTTP. Uma definição OpenAPI pode documentar endpoints, parâmetros, respostas e outras características da interface. A especificação também pode ser utilizada por ferramentas de documentação, geração de código e testes. A OpenAPI Initiative mantém a especificação oficial, cuja versão publicada mais recente é a 3.2.0.

6. Autenticação e segurança

Uma API exposta na Internet precisa controlar quem pode acessá-la e quais operações podem ser executadas. Entre os mecanismos utilizados estão API keys, tokens, autenticação baseada em Bearer Token, JWT e OAuth 2.0. A escolha depende do cenário e do nível de controle necessário.

OAuth 2.0 é um framework de autorização padronizado pelo IETF que permite que uma aplicação de terceiros obtenha acesso limitado a um serviço HTTP, inclusive em nome de um usuário, sem que a aplicação precise necessariamente receber as credenciais do usuário. O padrão define fluxos e papéis para a concessão de acesso.

Além da autenticação, boas práticas incluem utilizar HTTPS, validar entradas, limitar requisições (rate limiting), registrar eventos relevantes, evitar exposição de chaves secretas no código, controlar permissões, tratar erros adequadamente e manter as APIs atualizadas. O rate limiting evita que um cliente sobrecarregue o servidor com requisições excessivas, geralmente limitando a quantidade de chamadas por minuto ou hora.

Outro aspecto relevante em APIs Web é o CORS (Cross-Origin Resource Sharing), mecanismo que controla se um navegador pode fazer requisições para um domínio diferente daquele que serviu a página, sendo essencial para APIs consumidas por aplicações front-end hospedadas em outros domínios.

7. Exemplos práticos

7.1. API do Google Maps - Geocoding

Um exemplo de API utilizada em aplicações reais é a Geocoding API da Google Maps Platform. Ela permite converter endereços em coordenadas geográficas e também realizar o processo inverso. Assim, uma aplicação de entregas pode transformar um endereço em latitude e longitude para posicioná-lo em um mapa. A documentação oficial informa que a API oferece respostas em JSON ou XML e pode ser acessada por HTTP.

7.2. GitHub REST API

A GitHub REST API é uma API que permite criar integrações, consultar dados e automatizar fluxos de trabalho relacionados à plataforma GitHub. Entre outras possibilidades, uma aplicação pode consultar repositórios, issues, pull requests e informações de usuários. A documentação oficial também informa que a API é versionada e possui descrição compatível com OpenAPI.

7.3. APIs de pagamentos

Serviços de pagamento são um exemplo clássico de integração por API. Uma loja virtual pode enviar informações de uma compra para um provedor de pagamentos e receber uma resposta indicando se a operação foi aprovada, recusada ou ficou pendente. Diversos sistemas e empresas organizam suas APIs em torno de recursos REST, utilizando URLs orientadas a recursos, respostas em JSON e códigos de status HTTP.

8. Quadro comparativo entre os modelos de API

Modelo	Característica	Vantagem	Exemplo de uso
REST	Recursos e métodos HTTP	Simplicidade e grande adesão	Sistemas Web e mobile
SOAP	Mensagens XML e contrato formal	Padronização e interoperabilidade	Integrações corporativas
GraphQL	Cliente define os dados consultados	Flexibilidade nas consultas	Aplicações com telas variadas
WebSocket	Comunicação bidirecional persistente	Atualizações em tempo real	Chat e dashboards
Webhook	Notificação enviada por evento	Evita consultas repetitivas	Pagamentos e automações

9. Conclusão

Ao final da pesquisa, conclui-se que APIs e integrações são elementos fundamentais para o funcionamento dos sistemas modernos, pois permitem que aplicações diferentes compartilhem dados e funcionalidades de maneira controlada. A utilização de interfaces bem definidas reduz a dependência entre componentes e facilita a construção de sistemas compostos por serviços independentes.

Entre os principais modelos estudados estão REST, SOAP, GraphQL e WebSocket, além de mecanismos de integração como webhooks e polling. Não existe um único modelo ideal para todos os projetos: a escolha deve considerar os requisitos de desempenho, segurança, compatibilidade, volume de dados, necessidade de comunicação em tempo real e complexidade do sistema.

Também foi possível observar que ferramentas como Postman, OpenAPI, Insomnia e API Gateways auxiliam de diversas maneiras a utilização de uma API, desde o desenvolvimento e teste até a documentação, publicação e monitoramento. Exemplos como Google Maps, GitHub e plataformas de pagamento demonstram que as APIs estão presentes em serviços utilizados diariamente.

Portanto, compreender APIs não significa apenas saber fazer uma requisição HTTP. É necessário entender contratos, formatos de dados, autenticação, segurança, versionamento, tratamento de erros e estratégias de integração. O uso adequado desses conceitos contribui para sistemas mais interoperáveis, escaláveis, seguros e fáceis de manter.

10. Referências Bibliográficas

BIEHL, M. API Architecture - The Big Picture for Building APIs.

Disponível em:

<https://books.google.com.br/books?hl=pt-BR&lr=&id=6D64DwAAQBAJ&oi=fnd&pg=PA15&dq=API&ots=zbaM3rNh7s&sig=Qxx2tm3Ny2Pfdifoll6s8wnfX-w#v=onepage&q&f=false>

IBM. O que é uma API (Interface de Programação de Aplicativos)?

Disponível em: <https://www.ibm.com/br-pt/think/topics/api>

Amazon Web Services (AWS). API (interface de programação de aplicações).

Disponível em: <https://aws.amazon.com/pt/what-is/api/>

Sensedia. Tipos de APIs: Web, RESTful, SOAP, GraphQL e além.

Disponível em: <https://www.sensedia.com.br/pillar/tipos-de-apis>

Alura. Entenda o que é uma API e sua importância. Disponível em:

<https://www.alura.com.br/artigos/api?srsItid=AfmBOopHA5dhuHTRBHCR34xYrQloV VazvC8vHs0tiWdamiyyyjoPpqF9>

Google Cloud. Desenvolvimento de API: criação de serviços da Web escalonáveis.

Disponível em: <https://cloud.google.com/discover/what-is-api-development?hl=pt-BR>

Nota Gateway. O que é o Postman?

Disponível em: <https://notagateway.com.br/blog/postman/>